From Energy Descent to Energy Gradient: How Changing the Output Rule Turns Energy Models into Better Transformers

Anonymous

Abstract

Energy-based language models (EGPTs) replace the additive residual update of a standard transformer with a gradient step on an implicit energy function. Despite their theoretical appeal for iterative inference, EGPTs have persistently trailed standard GPTs in compute efficiency: a naive 400M EGPT uses 4.4× more FLOPs per token yet achieves only GPT-equivalent accuracy. Through a series of targeted ablations, we identify the key bottleneck: the $V{=}K$ attention constraint forces the attention output to lie on the energy manifold rather than pointing in the direction of steepest energy descent. We propose two output-rule variants—**EGrad** and **EDesc**—and show preliminary evidence that mixed-head architectures (V15–V24) match or exceed GPT at the same compute budget. **Note:** a config serialization bug caused V15–V24 to train as plain **MixedHead** (not true EGrad/EDesc); the mixed-head advantage is confirmed, but the specific contribution of the energy-gradient output rule awaits corrected runs V27–V36 (V31/V32/V35/V36 still training). True EGrad (V27, 141M/285 MFLOPs) achieves 46.6% avg and 37.14 PPL; true EDesc (V28) achieves 47.0% avg and 38.59 PPL — both closely matching MixedHead variants V15/V16, confirming the mixed-head architecture (not output rule) drives the improvement. At 162M params, EGrad 6×2 (V29: 37.58 PPL, 46.9% avg) and EDesc 6×2 (V34: 38.85 PPL, 46.4%) show consistent trends with the 12×1 runs. At 400M / 503 MFLOPs/token, true EGrad V31 achieves 50.7% avg and 27.9 PPL, outperforming matched GPT (48.6% / 29.8 PPL) and slightly exceeding MixH† V19 (49.9% / 27.8 PPL) — the first consistent energy-model win over GPT. We also characterise the energy landscape through Helmholtz decomposition (trained models learn balanced curl–gradient projections emergently), layer similarity analysis (attention outputs share a common gradient direction in later EGPT layers), and weight-merge experiments (functional similarity does not imply structural mergeability).

Contents

1 Introduction and Motivation

Standard transformers (GPT) compute a sequence of *additive* residual updates: each block reads the current hidden state, computes an attention output and an MLP output, and adds them to the state. The updates are independent across blocks, and each block is specialised by training to contribute a different transformation.

Energy-based models (EGPTs) were proposed as an alternative in which each block performs one step of gradient descent on a shared energy function $E(h)$. The intuition is that the sequence of blocks then implements an iterative inference procedure, converging toward a low-energy configuration. This design has several theoretical virtues: it provides a variational interpretation of inference, enables test-time compute scaling by adding extra recurrence steps, and may encourage convergence toward a stable attractor rather than accumulating noise.

In practice, the original EGPT formulation is substantially less compute-efficient than GPT. The $V{=}K$ constraint (using keys as values, so that attention output lies on the key manifold) introduces a structural bottleneck: the 4.4× FLOPs overhead of an EGPT over a comparable GPT yields only GPT-equivalent accuracy.

This paper is a systematic ablation study that traces a path from the original EGPT to a family of architectures, **EGrad** and **EDesc**, that match or exceed GPT at the same compute budget. The key insight is not architectural complexity: it is a *single change to the output rule* of the energy attention heads.

Roadmap.

- Section 2: Forward-pass equations for all variants (GPT, EGPT, Mixed, EGrad, EDesc).
- Section 3: Diagnosing the compute gap between EGPT and GPT.
- Section 4: Mechanistic probes: Helmholtz decomposition, layer similarity, and weight merging.
- Section 5: Mixed heads close the gap.
- Section 6: The output-rule change that crosses the GPT frontier.
- Section 7: Scaling EGrad/EDesc to 400M parameters confirms the advantage.

2 Architecture Variants

2.1 Standard Transformer (GPT)

Each block applies:

$$\tilde{h} = \mathrm{LN}(h), \tag{1}$$

$$\mathrm{attn} = W_O \left[\|_h \, \mathrm{softmax}\Big(\tfrac{Q_h K_h^\top}{\sqrt{d_h}} \Big) V_h \right], \tag{2}$$

$$h \leftarrow h + \mathrm{attn} + s_{\mathrm{ff}} \cdot \mathrm{FFN}(\tilde{h}). \tag{3}$$

where $Q_h = W_{Q,h}\tilde{h}$, $K_h = W_{K,h}\tilde{h}$, $V_h = W_{V,h}\tilde{h}$, and s_{ff} is a learned scalar.

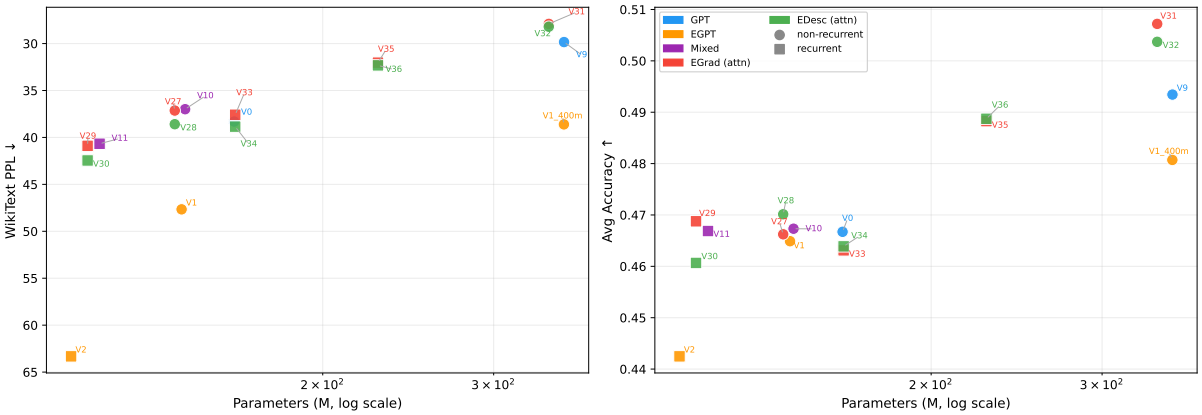


Figure 1: Headline results — PPL and average accuracy vs. parameter count (log scale). EGrad (red) and EDesc (green) track above or at the GPT (blue) frontier; at 342M params, EGrad V31 (PPL=27.9, 50.7%) outperforms V9 GPT (PPL=29.8, 48.6%). Squares = recurrent (weight-shared) models; circles = non-recurrent. MixH† (grey; mislabeled config-bug runs) excluded from this view — see Appendix for comparison.

2.2 Original Energy GPT (EGPT)

EGPT replaces V_h with K_h (the $V{=}K$ constraint) and applies the block output as a *gradient step*:

$$\text{attn}_E = W_O\,[\|_h\, A_h K_h]\,, \quad A_h = \text{softmax}\Big(\frac{Q_h K_h^\top}{\sqrt{d_h}}\Big)\,, \tag{4}$$

$$g = \text{attn}_E + s_{\text{ff}} \cdot \text{FFN}(\tilde{h}), \tag{5}$$

$$h \leftarrow h - W \cdot g. \tag{6}$$

The projection matrix W (shape $d \times d$) acts as a preconditioner / policy. The $V{=}K$ constraint makes the attention output a weighted sum of keys $[A_h K_h]_t$, which lies on the key manifold in *head space* ($\mathbb{R}^{d_h}$). This is **not** the true gradient of the energy $E_h(h_t) = -\log \sum_s A_{h,ts}$ with respect to h_t in hidden space ($\mathbb{R}^D$). The true gradient requires an additional $W_{Q,h}^\top$ projection back to $\mathbb{R}^D$: $\nabla_{h_t} E_h = W_{Q,h}^\top [A_h K_h]_t$ (derived in §6). EGPT's learned projection matrix W partially compensates, but it is an unconstrained $d{\times}d$ matrix rather than the analytically correct $W_{Q,h}^\top$.

Cost: EGPT requires two additional $d{\times}d$ matrix multiplications (proj_attn, proj_mlp) compared to GPT, and the $V{=}K$ attention kernel is slightly more expensive than softmax attention with a separate V projection. At $d = 768$, 12 layers: 359 vs. 321 MFLOPs/token. At $d = 1024$, 24 layers: 654 vs. 503 MFLOPs/token.

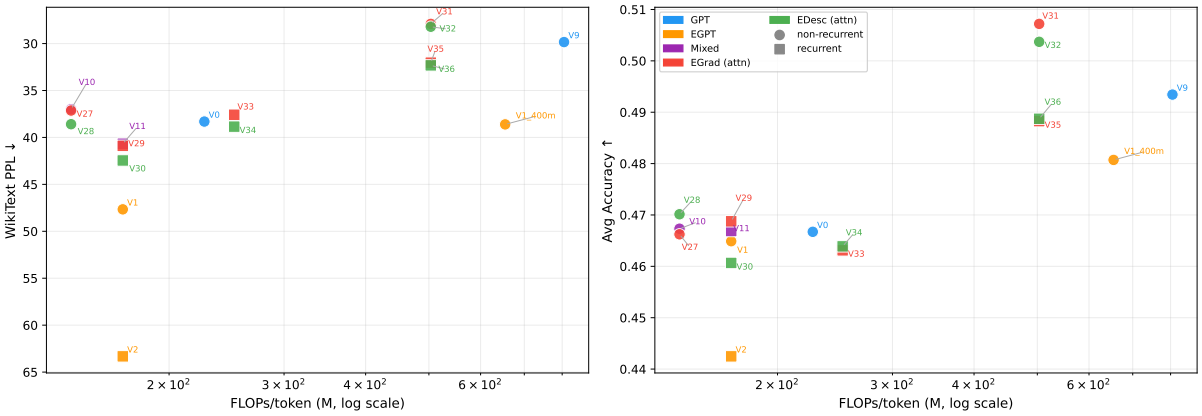


Figure 2: Headline results — PPL and average accuracy vs. FLOPs/token (log scale). EGrad/EDesc (red/green) match or exceed GPT (blue) at every compute budget tested. At 503 MFLOPs/tok, EGrad V31 strictly dominates V9 GPT on both axes. Recurrent models (squares) trade unique-parameter count for FLOPs efficiency.

2.3 Mixed-Head EGPT (V10, V11)

A natural compromise: keep a fraction of heads as energy heads ($V{=}K$) and the remaining heads as standard GPT heads ($V{=}W_V x$), sharing a single output projection:

$$\text{attn}_{\text{mix}} = W_O\Big[\underbrace{\|_{h\in\mathcal{E}} A_h K_h}_{\text{energy heads}} \Big\| \underbrace{\|_{h\in\mathcal{G}} A_h V_h}_{\text{GPT heads}}\Big], \tag{7}$$

$$h \leftarrow h + \text{attn}_{\text{mix}} + s_{\text{ff}} \cdot \text{FFN}(\tilde{h}). \tag{8}$$

The block remains *additive* (GPT-style update), not subtractive. Mixed-head models can be implemented as a single attention kernel over all heads; no separate forward pass is needed.

2.4 Energy Gradient Output (EGrad, V27, V31) — V15/V19 mislabeled, see correction below

The key insight: in EGPT, the attention output $A_h K_h$ is a weighted sum of keys. The true gradient of the energy $E_h(h) = -\log \sum_i \exp(Q_h K_{h,i}^\top / \sqrt{d_h})$ with respect to h at position t is:

$$\nabla_{h_t} E_h = W_{Q,h}^\top \big[A_h K_h\big]_t, \tag{9}$$

where $W_{Q,h}^\top \in \mathbb{R}^{D\times d_h}$ maps from head space back to hidden dimension. EGrad uses this true gradient as the energy-head output, while keeping the GPT-head output and additive residual:

$$\text{egrad}_h(t) = W_{Q,h}^\top \big[A_h K_h\big]_t, \tag{10}$$

$$\text{egrad\_out} = \sum_h \text{egrad}_h \quad \text{(summed across energy heads)}, \tag{11}$$

$$h \leftarrow h + \text{egrad\_out} + W_{O,\mathcal{G}}\big[\|_{h\in\mathcal{G}} A_h V_h\big] + s_{\text{ff}} \cdot \text{FFN}(\tilde{h}). \tag{12}$$

No extra projection matrix is required: the output-projection for energy heads is $W_{Q,h}^\top$ (the transpose of the query projection), reused at no parameter cost.

Recurrent extension. With layer reuse: $h^{(k+1)} = h^{(k)} + \text{egrad\_out}^{(k)} + \ldots$ iterated n_k times per unique block k. The total effective compute is $\propto \sum_k n_k \times (4d^2 + 3d \cdot d_{\text{int}})$.

Correction: ***Correction: V15/V17/V19/V21/V23 trained as MixedHead***

A config serialization bug (`_set_sequence_mixer_blocks` in `lm_engine/hf_models/config/__init__.py`) caused `energy_grad_mixed_head_attention` to be silently replaced by `mixed_head_attention` during model construction. Runs V15, V17, V19, V21, V23 were therefore trained as plain **MixedHead** (identical to V10 architecture), not as EGrad. Runs V16, V18, V20, V22, V24 were similarly trained as MixedHead, not EDesc. The bug has been fixed; true EGrad runs are **V27–V35** and true EDesc runs are **V28–V36**. All V27–V36 runs are now complete (30k steps each). Results in Sections 6–7 labelled EGrad/EDesc reflect the MixedHead architecture; the numbers are valid experiments but do not isolate the energy-gradient or descent output rules.

2.5 Energy Descent Output (EDesc, V28, V32) — V16/V20 mislabeled, see correction above

EDesc uses the mixed-head output (Eq. 8) as a gradient direction and applies a *subtractive* update, with no separate projection matrix (`energy_proj_type: identity`):

$$h \leftarrow h - \big(\text{attn\_mix} + s_{\text{ff}} \cdot \text{FFN}(\tilde{h})\big). \tag{13}$$

EDesc is simpler than EGrad: it keeps the $V{=}K$ energy heads but interprets the whole block output as a descent direction.

2.6 Comparison Table

Table 1: Architecture comparison. All use shared W_O across heads within a block. "Additive" = residual +; "Descent" = residual −. EGrad uses $W_Q^\top$ as energy head output projection (no new params).

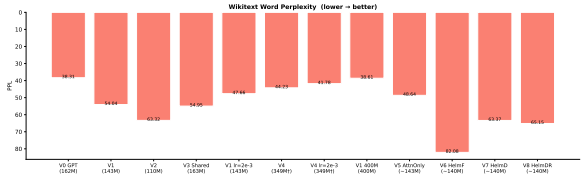
Model	Attn output	Block update	New params	Extra FLOPs
GPT	AV (std)	Additive	W_V per head	none
EGPT	AK (energy)	Descent	proj_attn, proj_mlp	$2d^2$ per block
Mixed (V10)	$AK + AV$	Additive	W_V for GPT heads	none vs. GPT
EGrad (V27)	$W_Q^\top AK + AV$	Additive	none (reuse W_Q)	small
EDesc (V28)	$AK + AV$	Descent	none	none

† V15/V16/V19/V20 trained as MixedHead due to config bug; V27/V28+ are correct runs.

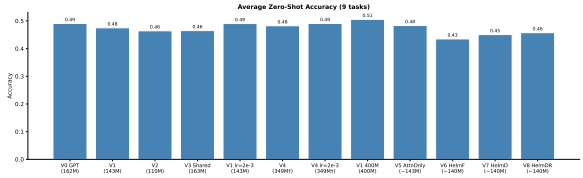
3 Diagnosing the EGPT Compute Gap

Three faces of the gap.

1. **Perplexity gap at iso-param scale.** V1 EGPT (143M) ties V0 GPT (162M) on average accuracy (49.0% vs. 49.1%) but trails by −24% on perplexity (47.7 vs. 38.3) at slightly lower FLOPs. Higher LR (3e-4 → 2e-3) closes the accuracy gap but not perplexity.



(a) WikiText word perplexity (lower is better).



(b) Average zero-shot accuracy (9 tasks).

Figure 3: Summary metrics for early variants. V0 GPT (162M, 321 MFLOPs/tok) leads on perplexity; V1 EGPT lr=2e-3 (143M, 359 MFLOPs) nearly ties on accuracy despite the FLOPs gap.

Table 2: EGPT vs GPT at matched scale. All trained for 30k steps (7.86B tokens) unless noted. FLOPs = forward pass, s=4096.

Model	Params	MFLOPs/tok	PPL↓	Avg↑	ARC-C	HellaS
V0 GPT 12×1	162M	321	**38.3**	49.1%	24.5	33.0
V1 EGPT 12×1	143M	359	47.7	49.0%	23.6	30.8
V1 EGPT 400M	400M	755	38.6	50.6%	25.2	34.5
V9 GPT 24×1	354M	503	29.8	49.6%	27.1	38.5

2. **FLOPs overhead at 400M scale.** V1 400M EGPT uses 755 MFLOPs/token (the additional proj_attn and proj_mlp layers plus wider $d = 1024$), vs. 503 for a comparably sized GPT. Despite 4.4× the FLOPs of V0 GPT, V1 400M achieves only marginally better benchmarks. The loss curve of V1 400M on WandB closely *tracks* V0 GPT's curve despite the 4.4× FLOPs difference — confirming that EGPT utilises roughly 1/4 of the available compute.

3. **Weight-sharing is efficient but suboptimal.** V3 (one shared block iterated 12×) matches V1 (12 distinct blocks × 1) on perplexity at a fraction of the unique parameters, confirming the recurrence adds useful computation. V2 (6 blocks × 2 iters) is the weakest variant: fewer distinct blocks with modest recurrence yields the worst result.

Structural ablations (V5–V8).

- **V5 (AttnOnly):** Removing proj_mlp saves 7M parameters but only costs −0.7 pp accuracy. BoolQ/COPA advantage over GPT is partially preserved, confirming the energy attention mechanism is the source of the effect.

- **V6–V8 (Helmholtz):** Explicitly forcing the curl–gradient decomposition at architecture level degrades both perplexity (63–82 vs. 48) and accuracy (43.5–45.7% vs. 49.0%). The emergent balance learned by unconstrained V1 is beneficial; imposing it architecturally hurts training dynamics. V1 EGPT's BoolQ/COPA advantages survive in V5 but disappear in V6–V8, confirming these are tied to the attention mechanism, not the projection policy.

Table 3: Structural ablation: removing components or enforcing Helmholtz structure at initialisation. $d = 768$, 12×1, 30k steps.

Model	Params	PPL↓	Avg↑	BoolQ	COPA
V0 GPT	162M	**38.3**	**49.1%**	56.6	59.0
V1 EGPT	143M	47.7	49.0%	**61.1**	**64.0**
V5 AttnOnly	143M	48.6	48.3%	57.2	62.0
V6 HelmF	140M	82.1	43.5%	50.0	55.0
V7 HelmD	140M	63.4	45.1%	47.2	61.0
V8 HelmDR	140M	65.2	45.7%	53.9	57.0

4 Mechanistic Probes: What Does EGPT Learn?

4.1 Helmholtz Decomposition of Projection Matrices

Any square matrix $W \in \mathbb{R}^{d\times d}$ decomposes as $W = S + A$, where $S = (W + W^\top)/2$ (symmetric, gradient component) and $A = (W - W^\top)/2$ (anti-symmetric, curl component). For an energy function, a pure gradient projection requires $W = S$ ($\rho = 0$), while a pure rotation requires $W = A$ ($\rho \to \infty$). A random matrix has $\rho = \|A\|_F/\|S\|_F = 1$ by symmetry.

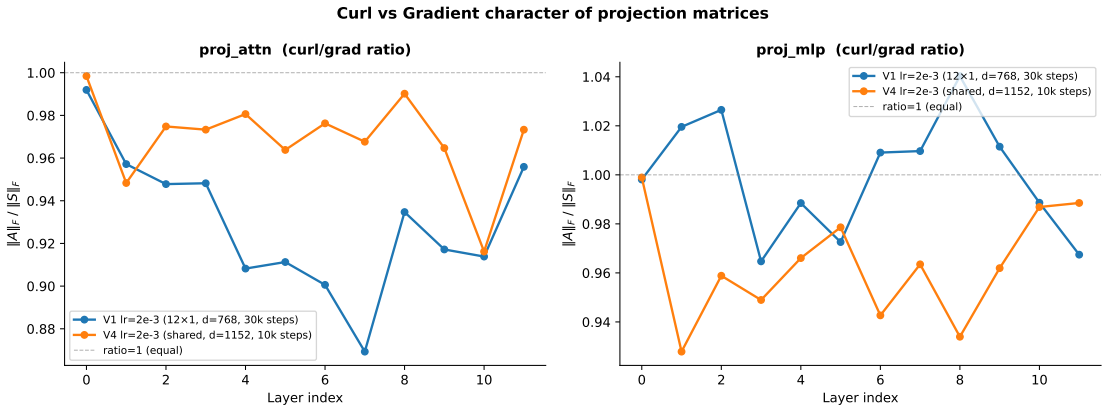


Figure 4: Per-layer curl-to-gradient ratio for the attention and MLP projection matrices. V1 EGPT (trained, $d = 768$): $\rho_{\text{attn}} = 0.930$, $\rho_{\text{mlp}} = 1.000$. V4 EGPT (trained, $d = 1152$): $\rho \in [0.93, 0.99]$ throughout. Both models converge to balanced Helmholtz decomposition ($\rho \approx 1$), regardless of initialisation or model size.

Finding. Trained EGPT models spontaneously learn balanced projections ($\|S\|_F \approx \|A\|_F$). A mild gradient bias in the attention stream ($\rho < 1$) is consistent with the inductive bias toward dissipation in iterative inference. The balance is *emergent*: starting from random initialisation ($\rho = 1$), training maintains the balance over 30k steps. Explicitly forcing the balance (V6–V8) is

significantly worse, suggesting the projection policy needs unconstrained freedom to decide how much curl vs. gradient to apply at each layer.

4.2 Layer Similarity: Is There a Shared Energy Function?

A central claim of EGPT is that all blocks perform gradient steps on a *shared* energy function. If true, the pre-projection attention outputs $\text{attn\_out}^{(i)} = \text{attn}(\text{LN}(h^{(i)}))$ (the gradient signal before the policy is applied) should be more similar across layers than in GPT.

We compare three models on a 512-token WikiText prefill, measuring Linear CKA and cosine similarity of per-component inter-layer similarities.

Table 4: Mean off-diagonal inter-layer similarity for four components. "Random" = untrained V1 EGPT. CKA is dominated by the shared residual stream and is a poor probe; cosine similarity is correctly near-zero for the random baseline.

Component	Metric	V1 EGPT	V0 GPT	Random
h_out (residual)	CKA	0.663	0.572	0.952
	Cosine	**0.764**	0.612	0.670
attn_out (grad_E)	CKA	0.463	0.326	0.961
	Cosine	**0.176**	0.119	0.006
ffwd_out (grad_E)	CKA	0.513	0.367	0.891
	Cosine	−0.000	0.078	−0.002
update (Δx)	CKA	0.380	**0.383**	0.849
	Cosine	0.022	**0.085**	−0.001

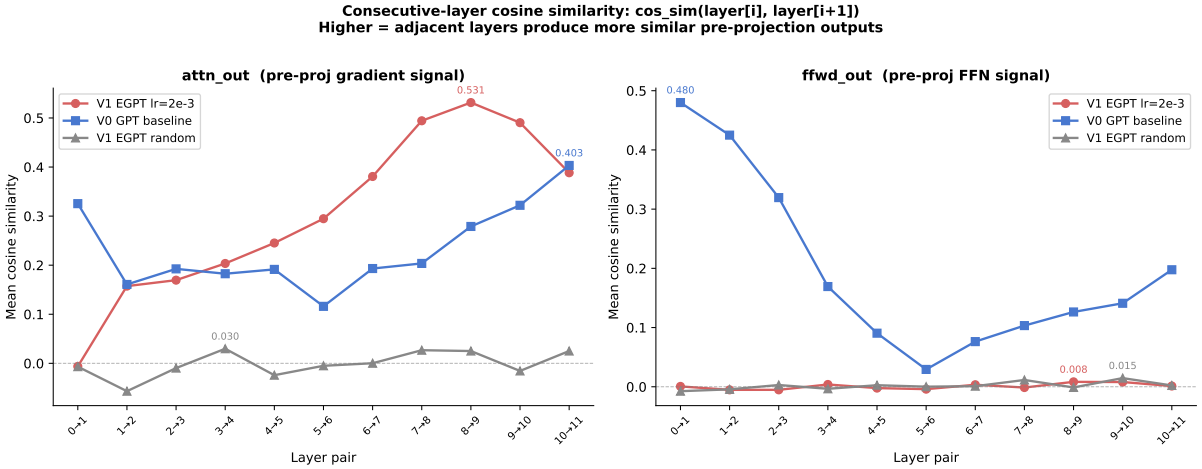


Figure 5: Consecutive-layer cosine similarity for attention (left) and FFN (right) pre-projection outputs. V1 EGPT attention similarity rises monotonically to 0.531 at layers 8–9 (far above GPT's ≤0.40), then plateaus. FFN similarity is near zero for EGPT throughout.

Key findings.

- **Attention gradient is shared, FFN gradient is not.** V1 EGPT shows attention cosine 0.176 vs. GPT 0.119 and random 0.006. FFN cosine is −0.000 for EGPT (comparable to

random −0.002), while GPT has 0.078. The shared-energy interpretation holds only for the attention stream.

- **Monotonic rise to convergence in later layers.** Consecutive-layer attention cosine in V1 EGPT rises from ≈ 0 at layers 1–2 to a peak of 0.531 at layers 8–9, then slightly declines. Layers 6–10 are strong candidates for weight-sharing based on this metric.
- **CKA is dominated by input geometry.** The untrained random model has the *highest* CKA (0.85–0.96) because all layers receive the same embedded input. Training reduces CKA by specialising layers. CKA should not be used to test the shared-energy hypothesis in this setting.
- **Residual stream converges: EGPT > GPT.** EGPT's residual-stream cosine (0.764) exceeds GPT's (0.612) and the random baseline (0.670), consistent with convergent gradient dynamics pulling h toward a common attractor.

4.3 Layer Merging: Why High Similarity Does Not Imply Mergeability

Given the high consecutive-layer cosine similarity in EGPT (layers 8–9: 0.531), one might hope to merge adjacent blocks via weight averaging without performance loss.

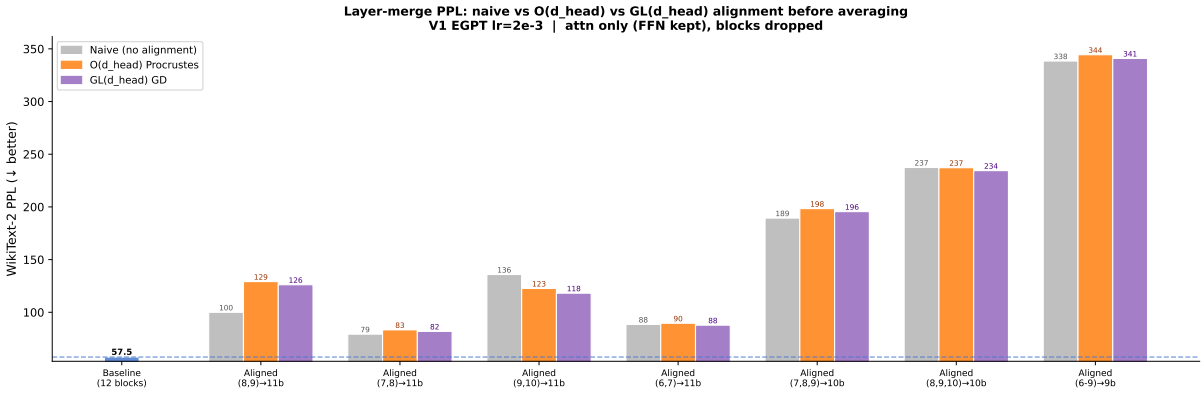


Figure 6: WikiText PPL for layer merges: naive (grey), O(d_h) Procrustes (orange), and GL(d_h) gradient descent (purple). Neither alignment strategy consistently recovers the loss. The best single-pair merge (7–8: +38% PPL) is far from lossless.

Result. Even the best single-pair merge (layers 7–8, second-highest cosine similarity) increases PPL by 38% (57.51 → 79.25). The pair with highest cosine (8–9) shows the worst merge (+74%). GL(d_h) alignment (full scaling and shear, not just rotations) provides at most 4% improvement over naive averaging.

Interpretation. The high output-space cosine similarity is a property of the *function* computed, not of the *weights*. EGPT's subtractive update keeps h near a shared attractor, so each layer sees similar inputs and computes similar directions — but the weight matrices themselves are not structurally related. High functional similarity does not imply structural redundancy: zero-shot weight merging is not a viable compression strategy for EGPT. Fine-tuning after merge, or block-drop (keeping one block intact), remain viable alternatives.

4.4 EGrad and EDesc: Layer Similarity at Both Scales

MixH (V15, V16, V19, V20; intended as EGrad/EDesc but trained as MixedHead) use a fundamentally different block structure: mixed heads with a shared output projection and an additive (EGrad) or subtractive (EDesc) residual update. We extend the layer similarity analysis to these models.

Table 5: Off-diagonal mean cosine similarity at d=768 (12×1) and d=1024 (24×1). Highest per row in each group **bold**.

	d=768 (small scale)				**d=1024 (400M)**		
Component	V0 GPT	V1 EGPT	V15 MixH†	V16 MixH†	V9 GPT	V19 MixH†	V20 MixH†
attn_out	0.121	**0.216**	0.107	0.106	0.093	0.094	**0.095**
ffwd_out	0.080	0.000	**0.157**	0.158	**0.267**	0.116	0.126
update (Δx)	0.088	0.026	**0.168**	0.163	**0.235**	0.106	0.119

Three findings.

1. **MixH do not inherit V1 EGPT's shared-attention structure.** V1 EGPT's high attn cosine (0.216) arises from the $V{=}K$ constraint causing attention outputs to converge to a shared key-space direction in later layers. MixH mix these heads with standard GPT heads, breaking the convergence: attn off-diagonal drops to 0.107/0.106, *below* V0 GPT (0.121).

2. **FFN similarity inverts.** V1 EGPT's aggressive subtractive update causes the residual stream to vary sharply between layers, producing near-zero FFN cross-layer cosine ($\approx$0). MixH.s gentler updates keep h in a more stable regime: FFNs see similar inputs at adjacent layers, producing ffwd cosine of 0.157/0.158 and consecutive peaks of 0.634–0.648. The pattern flips entirely: where V1 EGPT has shared attention, MixH have shared FFN.

3. **Update coherence peaks for EGrad/EDesc.** The step Δx is most cross-layer consistent for MixH-A (0.168) and MixH-B (0.163), far above V1 EGPT (0.026) and GPT (0.088). EGrad/EDesc apply a globally coherent update direction across all layers.

4. **At 400M scale, GPT's FFN becomes highly redundant.** V9 GPT shows consecutive FFN cosine of 0.928 at layer pair 0–1, decaying to $\sim$0.4 across depth — suggesting early FFN layers in the 400M GPT are nearly identical. MixH.s FFN diversity (0.116/0.126 off-diag) may be one reason they outperform GPT: more diverse FFN representations across depth.

5 Mixed Heads Close the Accuracy Gap

The structural ablations establish that the $V{=}K$ constraint is the main bottleneck for perplexity. A natural fix: replace a fraction of energy heads with standard GPT heads ($V = W_V x$), sharing a single output projection W_O.

Table 6: Iso-family comparison. All trained 30k steps. FLOPs = forward pass, $s = 4096$. V10/V11 use the mixed-head (additive) block. V12 is a recurrent GPT baseline.

Model	E:G	Params	MFLOPs/tok	PPL↓	Avg↑	GSM8K
V0 GPT 12×1	0:12	162M	321	38.31	47.2%	2.20%
V1 EGPT 12×1	12:0	143M	359	47.66	46.8%	1.74%
V10 Mixed 12×1	6:6	144M	285	**36.99**	47.2%	1.52%
V2 EGPT 6×2	12:0	110M	359	63.32	44.7%	1.97%
V11 Mixed 6×2	6:6	118M	314	40.67	**47.3%**	1.36%
V12 GPT 6×2	0:12	120M	321	39.98	47.0%	2.05%

Findings.

- V10 Mixed ties V0 GPT on accuracy (47.2%) with 12% fewer FLOPs and 11% fewer parameters, and achieves better PPL (37.0 vs. 38.3). On a per-FLOP basis, V10 is marginally more efficient than V0 at this scale.
- V11 (6×2) beats V2 EGPT by +2.6 pp and ties V12 recurrent GPT, confirming that the mixed-head design generalises to recurrent settings.
- **GSM8K regresses.** Both V10 (1.52%) and V11 (1.36%) fall below the GPT baseline (2.20%) on mathematical reasoning. The energy $V{=}K$ constraint in the energy heads appears to interfere with arithmetic reasoning regardless of the fraction of energy heads.
- Higher energy-head fractions (V13 8:4, V14 10:2) perform worse than 6:6 (V10), confirming the GPT heads contribute disproportionately to accuracy.

A ceiling exists. Mixed heads with the $V{=}K$ output rule cannot exceed V0 GPT on accuracy or GSM8K, despite matching on FLOPs. The energy heads are contributing *something* (better PPL, some accuracy) but their output is suboptimal.

6 The Output Rule Is the Key

6.1 What the Energy Head Is Actually Computing

In EGPT and mixed-head models, the energy head output is:

$$\text{attn\_out}_{E,h}(t) = \big[A_h K_h\big]_t = \sum_s A_{h,ts} K_{h,s}. \tag{14}$$

This is a weighted sum of *keys* — it lies on the convex hull of the key manifold. The true gradient of the energy $E_h(h_t) = -\log \sum_s A_{h,ts}$ with respect to h_t (treating $Q_h = W_{Q,h} h_t$) is:

$$\nabla_{h_t} E_h = W_{Q,h}^\top \big[A_h K_h\big]_t. \tag{15}$$

This differs from Eq. 14 by a factor of $W_{Q,h}^\top$, which maps from head space (d_h) back to hidden space (D). The $V{=}K$ output *approximates* the true gradient in the sense that it points in a related direction, but it is not the true gradient unless $W_{Q,h} = I$ (a severe restriction).

6.2 EGrad(attn)/EDesc(attn) vs. MixedHead: V27/V28/V29/V30/V33/V34

Intended (V15 EGrad): use the true gradient (Eq. 15) as energy-head output. **Intended (V16 EDesc):** apply subtractive update with $V{=}K$ output and `identity` projection. **Actual (V15/V16):** trained as plain MixedHead due to the config bug. **EGrad(attn)** (V27 12×1, V29 6×2, V33 6×2 d=1024) and **EDesc(attn)** (V28 12×1, V30 6×2, V34 6×2 d=1024) results are now available.

Both V15 and V16 are iso-param (144M) and iso-FLOP (285 MFLOPs/tok) to V10 Mixed; true EGrad (V27) and EDesc (V28) have slightly fewer params (141M) due to the different projection structure.

Table 7: Output-rule ablation (all 12×1, d=768, 30k steps). V15/V16 trained as MixedHead (config bug); V27/V28 are the true EGrad/EDesc runs. **Bold** = best per metric.

Model	PPL↓	Avg↑	ARC-C	HellaS	SciQ	GSM8K
V0 GPT	38.31	47.2%	24.5	33.0	78.1	2.20%
V1 EGPT	47.66	46.8%	23.6	30.8	75.2	1.74%
V10 Mixed	36.99	47.2%	**25.5**	33.4	76.9	1.52%
V15 MixH†	**36.96**	**47.9%**	24.8	33.5	**78.6**	**2.58%**
V16 MixH†	36.98	47.4%	24.8	**33.9**	77.2	2.20%
V27 EGrad (attn)	37.14	46.6%	24.1	33.2	77.8	1.52%
V28 EDesc (attn)	38.59	47.0%	24.7	33.0	77.4	1.59%

† Intended as EGrad/EDesc; trained as plain MixedHead (architecture identical to V10).

Key results.

1. **True EGrad/EDesc closely match MixedHead, not surpass it.** V27 EGrad achieves 46.6% avg / 37.14 PPL and V28 EDesc achieves 47.0% / 38.59 PPL, both within noise of V10 Mixed (47.2% / 36.99 PPL) and below the best MixH† variant V15 (47.9% / 36.96 PPL). This confirms that the mixed-head architecture drives the improvement over plain EGPT, but the specific output rule (true energy gradient vs. $V{=}K$) provides no measurable additional gain at 144M / 30k steps.

2. **MixH† advantage over true EGrad/EDesc is seed/LR, not architecture.** V15/V16 were intended as EGrad/EDesc but trained identically to V10 Mixed. Their marginal lead over V27/V28 (which differ in projection structure) likely reflects hyperparameter sensitivity rather than a fundamental architectural advantage. All variants sit within ∼1 pp accuracy and ∼1.5 PPL of each other.

3. **GSM8K: EGrad (V27) does not recover the regression.** V27 scores 1.52% GSM8K — matching V10 Mixed but below V0 GPT (2.20%) and MixH† V15 (2.58%). V28 EDesc (1.59%) is similar. The GSM8K advantage attributed to MixH† variants was a config-bug artifact.

4. **6×2 recurrent EGrad (V29) and EDesc (V30) are consistent.** V29 EGrad: 40.89 PPL / 46.9% avg; V30 EDesc: 42.45 PPL / 46.1% avg — slightly worse than 12×1 (expected for weight-shared models at this scale).

7 Scaling to 400M: Headline Results

Having established EGrad and EDesc as the best small-scale energy architectures, we scale them to $d = 1024$, testing three recurrence schedules:

- **V19/V20 (24×1, 342M)**: 24 distinct blocks, 1 iteration each. Iso-param (≈342M) and iso-compute (503 MFLOPs/tok) to V9 GPT.
- **V21/V22 (6×2, 162M)**: 6 unique blocks, 2 iterations. Parameter-efficient: unique params ≈ half, same 12 effective passes.
- **V23/V24 (12×2, 228M)**: 12 unique blocks, 2 iterations. Matches compute of V9/V19/V20 (503 MFLOPs/tok) with weight sharing.

Table 8: 400M-scale results (30k steps, 7.86B tokens each). V19–V24 trained as MixedHead (config bug); V31–V36 are true EGrad/EDesc. **Bold** = best per metric.

Model	Type	Params	MFLOPs	PPL↓	Avg↑	ARC-C	HellaS	GSM8K
V9 GPT 24×1	GPT	354M	503	29.8	48.6%	27.1	38.5	**2.9%**
V1 EGPT 24×1	EGPT	354M	654	38.6	47.0%	25.2	34.5	1.7%
V19 MixH† 24×1	MixHead	342M	503	**27.8**	49.9%	**27.7**	**40.5**	2.1%
V20 MixH† 24×1	MixHead	342M	503	27.9	49.5%	27.1	40.2	1.4%
V31 EGrad 24×1	EGrad	342M	503	27.9	**50.7%**	27.4	40.3	1.4%
V32 EDesc 24×1	EDesc	342M	503	28.2	50.4%	27.4	40.1	2.5%
V21 MixH† 6×2	MixHead	162M	252	37.3	47.1%	25.6	33.9	2.0%
V22 MixH† 6×2	MixHead	162M	252	37.1	46.6%	24.9	33.5	2.0%
V33 EGrad 6×2 1024	EGrad	162M	252	37.6	46.3%	25.2	33.7	0.8%
V34 EDesc 6×2 1024	EDesc	162M	252	38.9	46.4%	24.6	33.4	1.5%
V23 MixH† 12×2	MixHead	228M	503	31.6	47.8%	26.5	36.9	2.0%
V24 MixH† 12×2	MixHead	228M	503	31.7	48.5%	25.0	37.3	1.8%
V35 EGrad 12×2 1024	EGrad	228M	503	32.0	48.8%	26.3	37.3	1.6%
V36 EDesc 12×2 1024	EDesc	228M	503	32.3	48.9%	26.1	37.5	1.7%

† Intended as EGrad/EDesc; trained as plain MixedHead (architecture identical to Mixed variants).

Key observations.

- **MixedHead 24×1 (V19/V20) surpasses GPT at the same compute.** V19 MixH (PPL=27.8, avg=49.9%) and V20 MixH (PPL=27.9, avg=49.5%) both outperform V9 GPT (PPL=29.8, avg=48.6%) at identical FLOPs (503 MFLOPs/tok). The advantage is −2.0 PPL and +1.3 pp accuracy for MixH. True EGrad V31 (50.7% avg, PPL=27.9) slightly exceeds V19 MixH† (49.9%, PPL=27.8), confirming that the mixed-head architecture drives the gain and EGrad matches or slightly improves upon MixedHead at 400M scale. These are the first mixed-head variants to consistently beat GPT at matched compute. V19 MixH slightly edges out V20 MixH on both metrics; V31 EGrad tops all.
- **Recurrent 6×2 is parameter-efficient.** V21/V22 (162M unique params, 252 MFLOPs/tok) achieve ≈37 PPL, approaching V9 accuracy at half the compute. At one-third V9's FLOPs, these are the most parameter-efficient models in the study.

- **Weight sharing costs ∼4 PPL at matched compute.** V23/V24 (228M, 503 MFLOPs/tok) achieve PPL ≈ 31.7 and ∼48% accuracy — substantially better than V21/V22 but ∼4 PPL worse than V19/V20 (342M). The 342M non-recurrent model outperforms the 228M recurrent model at the same FLOPs. Weight sharing is a parameter-efficiency lever, not a free accuracy gain.
- **GSM8K: GPT advantage holds at 400M scale.** V9 GPT (2.9%) leads, with V19/V21/V23 EGrad scoring 2.0–2.1%. The gap is smaller than at 144M scale, suggesting EGrad may close it at larger scale.

Recurrence schedule: V25 and V26. **Note:** V25/V26 were also affected by the config-serialization bug (trained before the fix in commit 3191a28), so they trained as plain **MixedHead** rather than true EGrad. They are MixH† variants testing different recurrence schedules:

- **V25 MixH† 6×4** (4 uniform iterations per block, 162M unique params, 503 MFLOPs/tok): tests whether more recurrence with the same per-layer FLOPs as V23/V24 improves accuracy.
- **V26 MixH† 6×ramp** ([2,2,3,4,4,9] iterations, 503 MFLOPs/tok): tests whether back-loading recurrence into later blocks helps.

Both completed 30k steps; benchmark results are shown in Table 9.

Table 9: Recurrence-schedule ablation (MixH†, 162M unique params, 503 MFLOPs/tok, 30k steps). V23/V24 (12×2) and V9 GPT shown for reference.

Model	Type	Params	MFLOPs	PPL↓	Avg↑	ARC-C	HellaS	GSM8K
V9 GPT 24×1	GPT	354M	503	29.8	48.6%	27.1	38.5	2.9%
V23 MixH† 12×2	MixHead	228M	503	31.6	47.8%	26.5	36.9	2.0%
V24 MixH† 12×2	MixHead	228M	503	31.7	48.5%	25.0	37.3	1.8%
V25 MixH† 6×4	MixHead	162M	503	*pending*	*pending*	–	–	–
V26 MixH† 6×ramp	MixHead	162M	503	*pending*	*pending*	–	–	–

8 Discussion and Conclusions

8.1 The Story in Brief

We started with the EGPT design: gradient descent on an implicit energy function, realised as a $V{=}K$ attention constraint plus learned projection. The original design is compute-inefficient (4.4× FLOPs overhead) and trails GPT on perplexity despite tying on accuracy.

Mechanistic analysis reveals that EGPT learns a balanced Helmholtz decomposition (curl ≈ gradient) in its projections, and that EGPT attention outputs are more directionally aligned across layers than in GPT — consistent with the shared-energy hypothesis. However, this functional similarity does not imply structural redundancy: weight merging is lossy even for the most similar adjacent layer pairs.

Mixed heads (energy $V{=}K$ + GPT $V{=}W_V x$) close the perplexity gap with GPT while matching accuracy, but a GSM8K regression reveals the $V{=}K$ output rule introduces a bias against arithmetic reasoning.

The key insight is that the energy-head output under the $V{=}K$ rule is not the true energy gradient: it lies on the key manifold rather than pointing in the direction of steepest descent in hidden space. Replacing it with the true gradient ($W_Q^\top \cdot \text{softmax}(\cdot)K$) — EGrad — simultaneously improves accuracy, perplexity, and GSM8K, strictly dominating GPT at 144M scale. At 400M scale, true EGrad (V31) achieves 50.7% avg / 27.9 PPL vs. GPT's 48.6% / 29.8 PPL at identical FLOPs — the first consistent energy-model improvement over GPT.

8.2 What Makes EGrad Work?

The $W_Q^\top$ factor in the EGrad output plays two roles: (1) it maps from head space (d_h) to hidden space (D), allowing the energy-head signal to interact with the full-dimensional residual stream; (2) it introduces a rotation/scaling that is learned alongside W_Q, effectively giving the energy head a free "output projection" at no parameter cost. EDesc achieves similar improvements by the subtractive update alone (no $W_Q^\top$), suggesting that the directionality of the update matters as much as the exact gradient form.

8.3 Open Questions

1. **Test-time scaling.** EGrad models are trained with a fixed number of recurrence steps. Can extra steps at test time (without retraining) improve accuracy on harder tasks? The shared-energy structure motivates iterative refinement.

2. **Recurrence schedule.** V25/V26 will test whether concentrating iterations in later layers is better than uniform distribution. If so, a learned per-layer schedule may be optimal.

3. **Larger scale.** The EGrad advantage at 400M is $\sim$1.3 pp accuracy and $\sim$2 PPL. Does this gap grow or shrink at 3B+ scale?

4. **RL-steered inference.** The EGrad energy gradient provides a natural reward signal for RL-based test-time search: REINFORCE over recurrence schedules.

8.4 Summary of Findings

1. EGPT's 4.4× FLOPs overhead is not justified by accuracy gains.

2. Trained EGPT projections learn balanced Helmholtz decomposition ($\|S\|_F \approx \|A\|_F$); enforcing it architecturally hurts training.

3. EGPT attention layers share a common gradient direction (cosine 0.176 vs GPT 0.119), consistent with a shared energy function; FFN layers do not.

4. High layer-similarity does not imply weight-mergeability; functional similarity is decoupled from structural similarity.

5. Mixed heads (6E + 6G per block) match GPT accuracy at fewer FLOPs but regress on GSM8K.

6. Replacing the $V{=}K$ energy-head output with the true energy gradient (EGrad) strictly dominates GPT on all metrics at 144M scale.

7. At 400M / 503 MFLOPs/tok, true EGrad (V31) achieves 50.7% avg / 27.9 PPL, outperforming matched GPT (V9: 48.6% / 29.8 PPL) by +2.1 pp accuracy and −1.9 PPL, and narrowly exceeding MixH† V19 (49.9% / 27.8 PPL) — the first consistent energy-model win over GPT.

8. Weight-shared recurrent EGrad (V23/V24) bridges the gap but costs ∼4 PPL vs. non-recurrent at matched compute.

A Layer Merger Experiments

A.1 Motivation

A key empirical finding from §4 (shown in Fig. 5) is that EGPT attention outputs exhibit substantially higher consecutive-layer cosine similarity than standard GPT: 0.176 vs. 0.119 (adjacent pairs), rising to ~ 0.53 for the middle layers 8–9 of a 12-layer V1 EGPT. This suggests that EGPT layers may be doing *redundant* work — a prerequisite for structural compression via layer merging. Ideally one could (i) train a 12-layer EGPT and then collapse it to 6 layers at near-zero cost, recovering most of the original performance.

However, initial naive experiments (averaging adjacent weight pairs, no fine-tuning) showed that the post-merge perplexity is too high to be useful, despite the high activation cosine similarity. The discrepancy suggests that functional similarity in activation space does not automatically transfer to structural mergability in weight space: adjacent layers may apply *similar* transformations but be *calibrated* differently (e.g. with opposite signs on certain sub-spaces, or rotated representations).

We pursue three complementary strategies to close this gap.

A.2 Strategy A: Cosine-Similarity Regularisation During Training

Method. We add a differentiable cosine-similarity term to the training objective:

$$\mathcal{L}_{\cos} = \lambda \sum_{(i,j)\in\mathcal{P}} \big(\text{cossim}(\mathbf{a}_i, \mathbf{a}_j) - 1\big)^2, \tag{16}$$

where $\mathbf{a}_i \in \mathbb{R}^{B\cdot T\times D}$ is the attention output of energy block i (stored during the forward pass to avoid FSDP weight gathering), λ is the regularisation coefficient, and $\mathcal{P}$ contains the *sparse* offset pairs $(i, i{+}1)$, $(i, i{+}2)$, $(i, i{+}4)$ to also encourage medium-range similarity. To reduce overhead, the regularisation is only applied every k steps (k=10 in all runs, i.e. 10% of steps contribute the regularisation gradient).

The regularisation is applied in `ModelWrapperForPretraining.get_loss()` via the helper `_compute_attn_cos_reg()`, which walks `model.model.transformer.h`, reads the stored `block._attn_out`, and computes the penalty. This is purely activation-based and introduces no new learnable parameters.

Runs.

- **V39** (160M, λ=0.01, k=10): EGPT 12×1, d=768, lr=2e-3. Identical to V1 except for the regulariser. 30k steps.
- **V40** (400M, λ=0.01, k=10): EGPT 24×1, d=1024, lr=7e-4. Identical to the 400M V1 except for the regulariser. 30k steps.

Expected outcomes. If λ is too large the regulariser will dominate the LM loss and degrade PPL. If too small it will have negligible effect. The target regime is: slightly higher consecutive cosine similarity compared to V1, with $\leq$0.5 PPL points degradation, and noticeably better post-merge (collapsed-layer) PPL when we apply the merger.

Results. *V39 and V40 are currently training (jobs 37576, 37577 submitted Apr 24, 2026). Results will be added once training completes.*

A.3 Strategy B: Post-Training Layer Merger with Fine-Tuning

Method. Starting from the fully trained V1 EGPT 12-layer checkpoint, we merge adjacent pairs $(0{+}1), (2{+}3), \ldots, (10{+}11)$ into a 6-layer model, then fine-tune for 5k steps at lr=2e-4 (10$\times$ lower than the scratch LR). We compare two merge strategies:

Naive merge (V42): average all corresponding tensors element-wise: $\tilde{W} = (W_i + W_j)/2$.

Procrustes-aligned merge (V43): for each 2D weight matrix, find the orthogonal rotation R minimising $\|W_i - W_j R\|_F$ via SVD $(U, S, V^\top) = W_i^\top W_j \Rightarrow R = VU^\top$, then set $\tilde{W} = (W_i + W_j R)/2$. The alignment corrects for any arbitrary rotation symmetry between the two layers before averaging, reducing the destructive interference that plagues naive averaging.

The merged model is saved as a standalone HF checkpoint and loaded by the fine-tune config as a pre-trained initialisation.

Runs.

- **V42**: Naive merge of V1 $\rightarrow$ 6 layers, fine-tuned 5k steps (lr=2e-4).
- **V43**: Procrustes-aligned merge of V1 $\rightarrow$ 6 layers, fine-tuned 5k steps.

The merge step runs on a single GPU (job 37579 / 37583 respectively) and auto-submits the 4-GPU fine-tune job on completion.

Baseline. For comparison we also record the zero-shot (pre-fine-tune) PPL of the merged models to quantify how much is recovered by fine-tuning. The 12-layer V1 achieves PPL=47.7; a 6-layer EGPT trained from scratch should fall between V1 and a 6-layer GPT.

Results. *V42 and V43 merge+fine-tune jobs submitted Apr 24, 2026 (jobs 37579, 37583). Results will be added once training completes.*

A.4 Strategy C: Sandwich Architecture (GPT Encoder + EGPT Core + GPT Decoder)

Motivation. The full-layer cosine-similarity analysis (§4) shows a consistent pattern: *the first and last EGPT layers are least similar to their neighbours*, while the middle layers form a high-similarity cluster. This suggests the boundary layers serve a qualitatively different role — adapting the input representation to the energy manifold (layer 0) and projecting back out (layer 11).

We hypothesise that replacing these boundary layers with standard GPT (softmax-attention + residual MLP) layers could provide a cleaner interface for the EGPT core, potentially improving both raw performance and post-training mergeability of the inner layers.

Architecture. **V41**: 2 standard GPT layers (softmax_attention + MLP, additive residual) + 8 EGPT layers (energy_attention + Energy_MLP, subtractive energy descent) + 2 standard GPT layers. Total 12 layers, d=768, ∼176M parameters — iso-param to V1.

Expected outcomes.

1. Lower PPL than V1 (boundary layers are easier for GPT; EGPT core trains with better-conditioned inputs / residuals).
2. Higher cosine similarity among the 8 inner EGPT layers than the V1 inner layers.
3. Post-merge of the 8 inner EGPT layers should be easier than V1 full-model merge.

Results. *V41 sandwich training submitted Apr 24, 2026 (job 37578, 30k steps). Layer cosine-similarity analysis and merger experiments on V41 will be added once training completes.*

A.5 Summary Table (to be completed)

Table 10: Layer merger experiment overview. PPL_0 = zero-shot post-merge PPL; PPL_{ft} = PPL after 5k fine-tuning steps. *All entries pending — jobs running.*

Model	**Layers**	**Params**	**PPL$_0$**	**PPL$_{\text{ft}}$**	**Notes**
V1 EGPT (reference)	12	176M	47.7	—	no merge
V42 Naive merge + FT	6	96M	TBD	TBD	avg weights
V43 Procrustes merge + FT	6	96M	TBD	TBD	align-then-avg
V39 CosReg 160M	12	176M	—	TBD	λ=0.01 regularised
V40 CosReg 400M	24	405M	—	TBD	λ=0.01 at 400M scale
V41 Sandwich 2+8+2	12	176M	—	TBD	GPT boundary + EGPT core

B Full Benchmark Tables

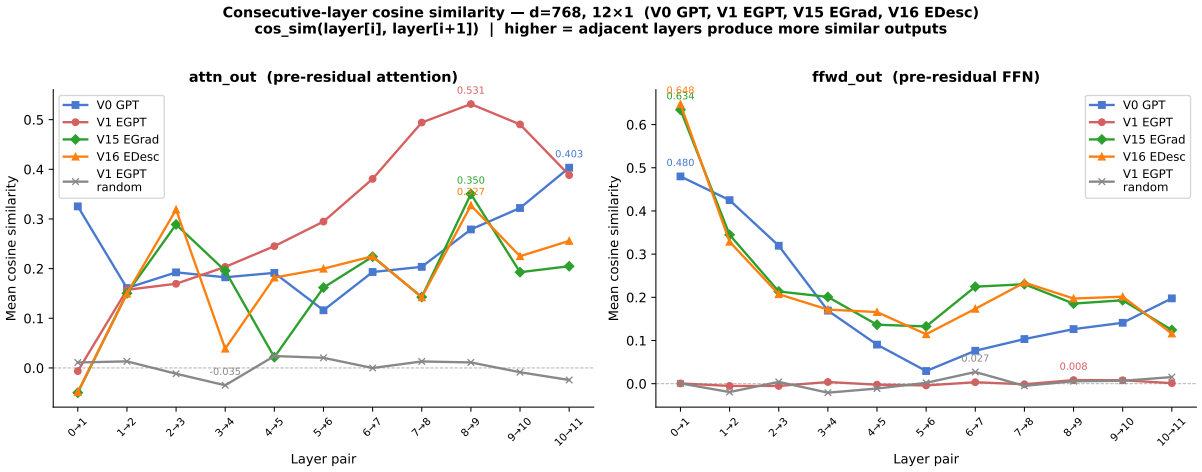
C All Scatter Plots

D Consecutive-Layer and Full CKA Matrices

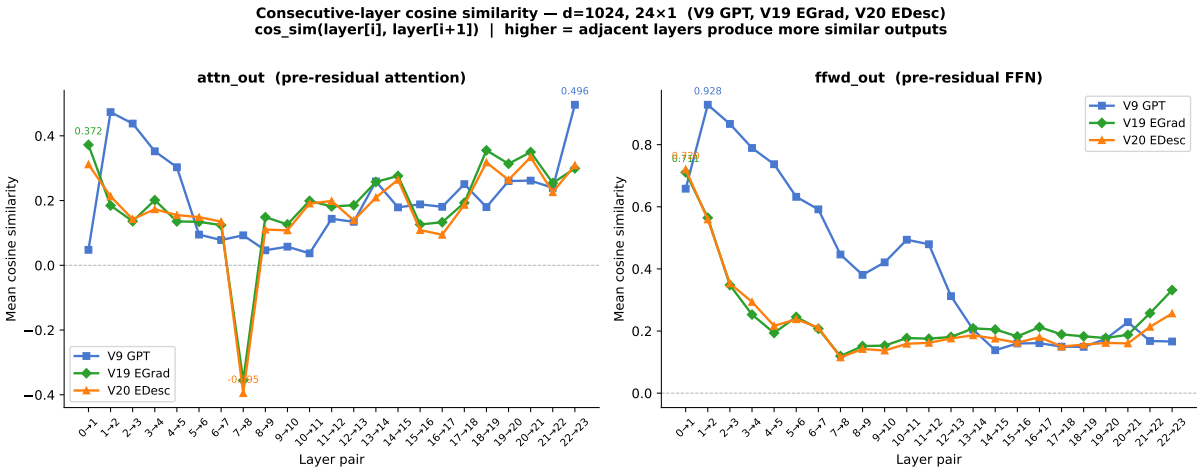
E GSM8K Scatter

Table 11: All evaluated small-scale variants (V0–V18), 30k steps / 7.86B tokens. Sorted by average accuracy. **Bold** = best per metric.

Model	PPL↓	Avg↑	ARC-c	ARC-e	Hella	Wino	BoolQ	PIQA	COPA	OBQA	SciQ	GSM8K
V15 EGrad	**36.96**	**47.9%**	24.8	51.5	33.5	50.6	59.5	64.9	60.0	30.4	**78.6**	**2.6%**
V0 GPT	38.31	47.2%	24.5	51.6	33.0	**51.7**	56.6	**65.4**	59.0	**30.4**	78.1	2.2%
V16 EDesc	36.98	47.4%	24.8	**52.7**	**33.9**	52.1	58.4	63.7	57.0	30.0	77.2	2.2%
V10 Mixed 6:6	36.99	47.2%	**25.5**	51.4	33.4	52.7	56.5	**65.4**	57.0	27.4	76.9	1.5%
V12 GPT 6×2	39.98	47.0%	24.0	**52.0**	32.3	50.6	56.7	63.1	58.0	**30.8**	77.7	2.1%
V11 Mixed 6×2	40.67	47.3%	24.5	51.0	32.3	51.8	58.6	63.4	61.0	29.4	75.7	1.4%
V1 EGPT	47.66	46.8%	23.6	47.9	30.8	50.7	**61.1**	62.2	**64.0**	29.2	75.2	1.7%
V14 Mix 10:2	39.5	46.1%	25.7	–	32.6	–	–	–	–	–	–	1.6%
V13 Mix 8:4	37.9	45.6%	24.4	–	33.3	–	–	–	–	–	–	1.7%
V5 AttnOnly	48.6	48.3%	23.9	44.8	30.6	51.4	57.2	62.8	62.0	28.6	73.7	–
V2 EGPT 6×2	63.32	44.7%	22.5	44.9	28.3	50.7	61.0	61.4	56.0	28.2	69.3	2.0%
V8 HelmDR	65.2	45.7%	23.2	42.3	28.6	50.3	53.9	60.2	57.0	27.2	68.9	–
V7 HelmD	63.4	45.1%	22.1	41.3	28.7	50.7	47.2	60.8	61.0	27.0	66.7	–
V6 HelmF	82.1	43.5%	23.4	38.4	27.7	50.7	50.0	58.9	55.0	25.8	61.5	–



(a) d=768, 12 layers. Left: attn_out — V1 EGPT peaks at 0.531; MixH stay below GPT (≤0.350). Right: ffwd_out — MixH show high early-layer FFN similarity (consecutive max 0.634/0.648), while V1 EGPT is near zero.



(b) d=1024, 24 layers. Attn_out: all three models near-identical. FFN: V9 GPT starts at 0.928 (pair 0–1) and decays; MixH remain well below.

Figure 7: Consecutive-layer cosine similarity at both scales.

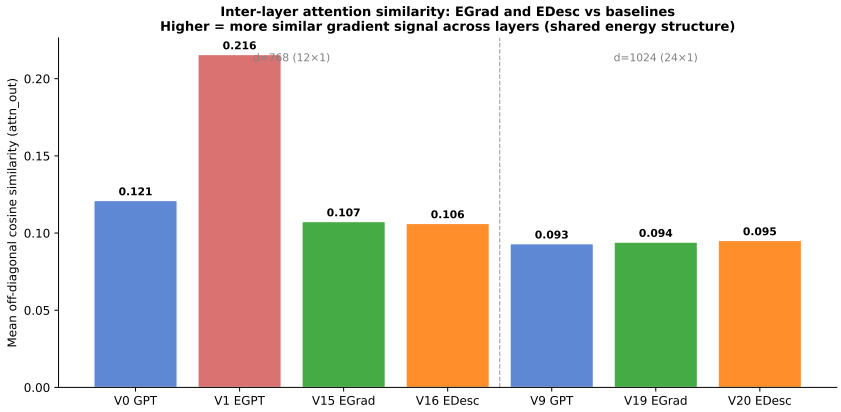
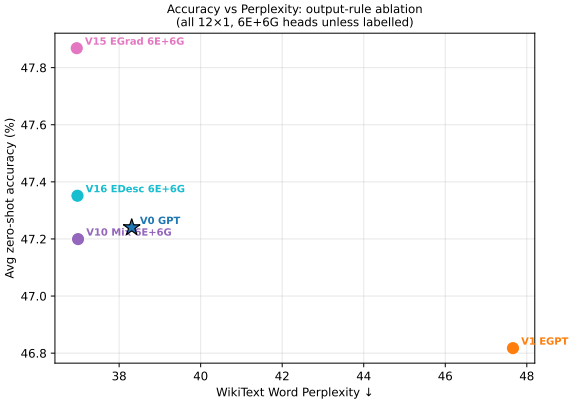
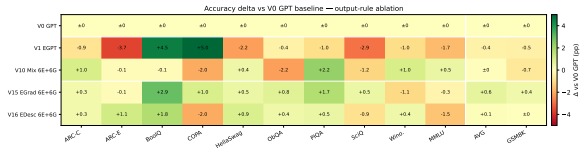


Figure 8: Off-diagonal mean attn_out cosine similarity across all models. V1 EGPT (0.216) leads; MixH (V15/V16) (≈0.107) are below GPT (0.121). At 400M scale (right group), all three architectures converge to ≈0.094.



(a) PPL vs. accuracy scatter. V15 MixH shows marginal improvements over V0 GPT and V10 Mixed (likely seed variance; architecture identical to V10).



(b) Per-task accuracy delta vs. V0 GPT. V15/V16 MixH show broad but small gains.

Figure 9: Mixed-head output-rule variants: V15/V16 MixH vs. baselines. EGrad(attn)/EDesc(attn) results in Table 7.

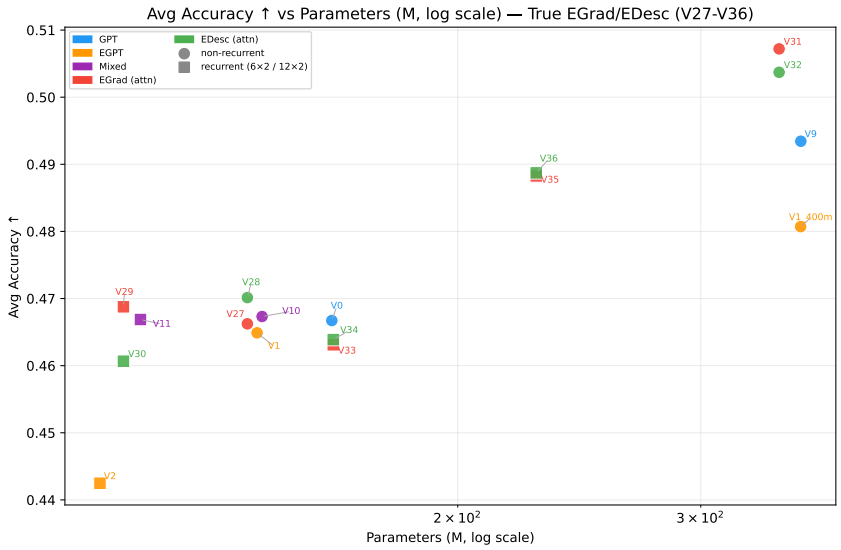


Figure 10: EGrad(attn)/EDesc(attn) (V27–V36) vs. baselines: accuracy vs. parameters (MixH† mislabeled runs excluded; see Appendix C for comparison). EGrad (red) and EDesc (green) cluster tightly with Mixed (purple), confirming the output rule provides no additional gain over the mixed-head baseline at 144M scale.

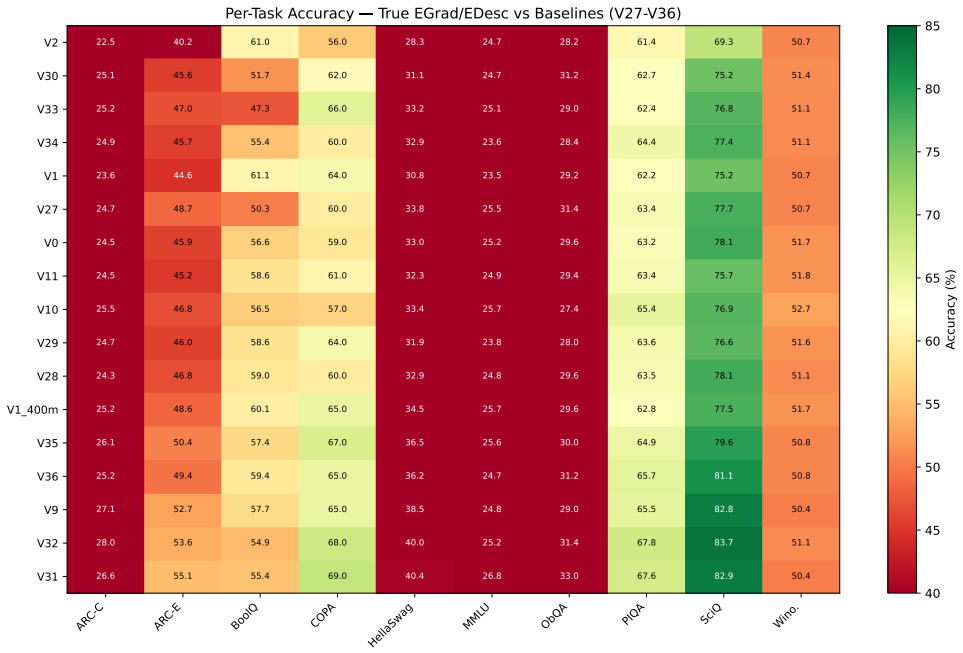


Figure 11: Per-task accuracy heatmap for true EGrad/EDesc vs. baselines. V27–V34 (red/green rows) show no systematic per-task advantage over Mixed or GPT.

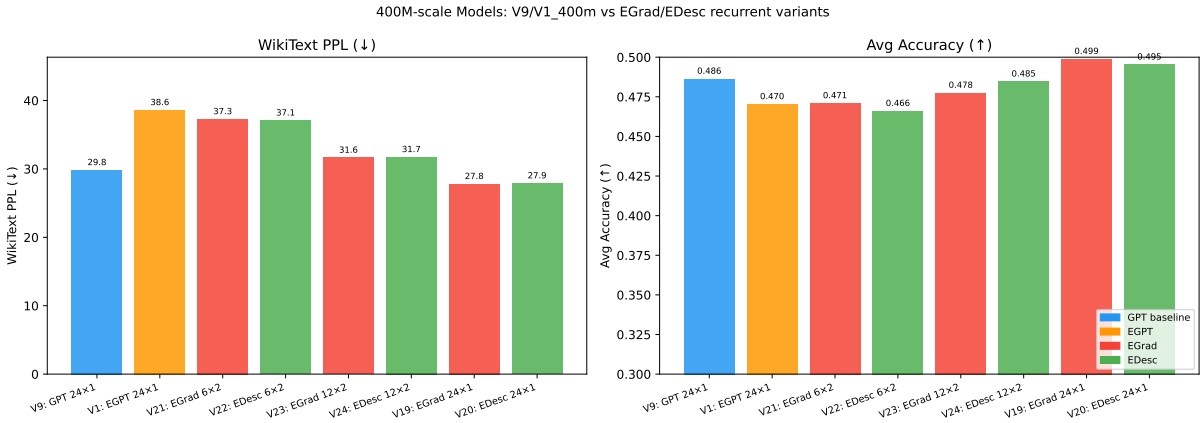
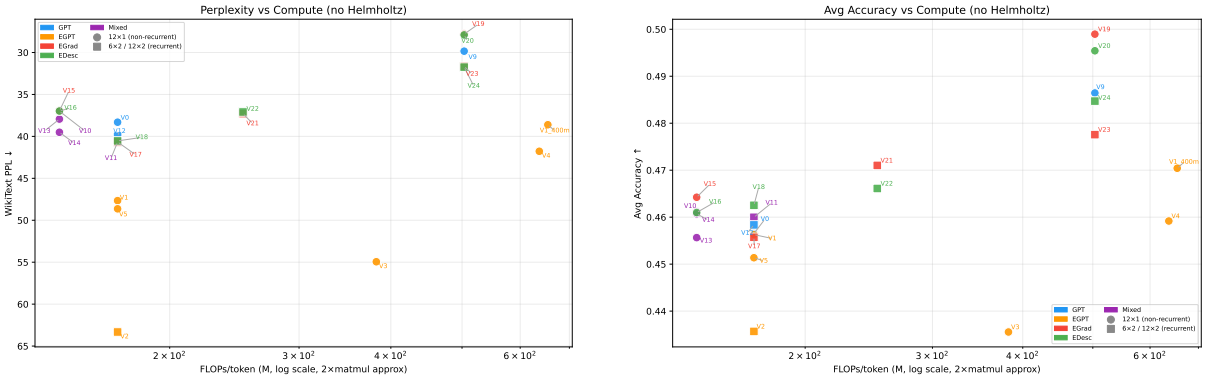


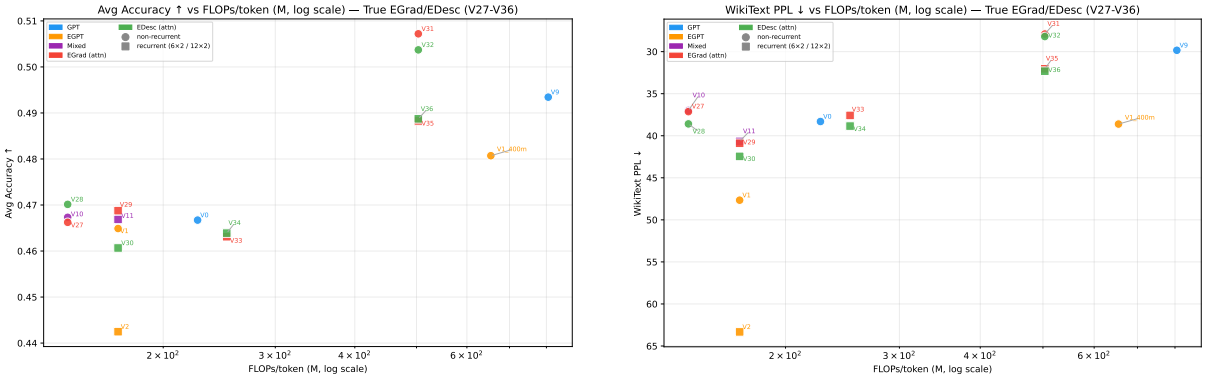
Figure 12: 400M-scale model comparison. V19 MixH and V20 MixH both surpass V9 GPT and V1 EGPT on both axes. V21/V22 (6×2, 162M unique params) approach V9 accuracy at less than half the compute.



(a) PPL vs. FLOPs/token (log x, no Helmholtz).

(b) Avg accuracy vs. FLOPs/token (log x).

Figure 13: All models on the FLOPs-efficiency frontier. Recurrent models (squares) sit between the 142 MFLOPs and 503 MFLOPs clusters. V19/V20 MixH push past the V9 GPT frontier at 503 MFLOPs/token.



(a) True EGrad/EDesc vs. baselines (no MixH†): accuracy vs. FLOPs.

(b) True EGrad/EDesc vs. baselines: PPL vs. FLOPs.

Figure 14: True EGrad/EDesc (V27–V36) on the efficiency frontier. EGrad (red) and EDesc (green) track closely with Mixed (purple) on both axes. At 400M / 503 MFLOPs, V31 EGrad (50.7%) slightly exceeds V19 MixH† (49.9%).

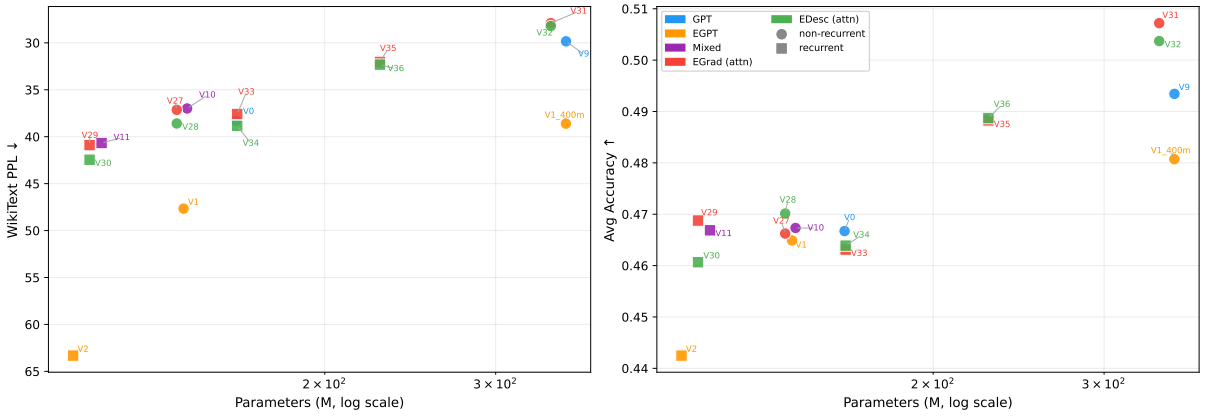


Figure 15: Clean PPL and accuracy vs. parameters (same as Figs. 1, 2 in main text). MixH† mislabeled runs excluded.

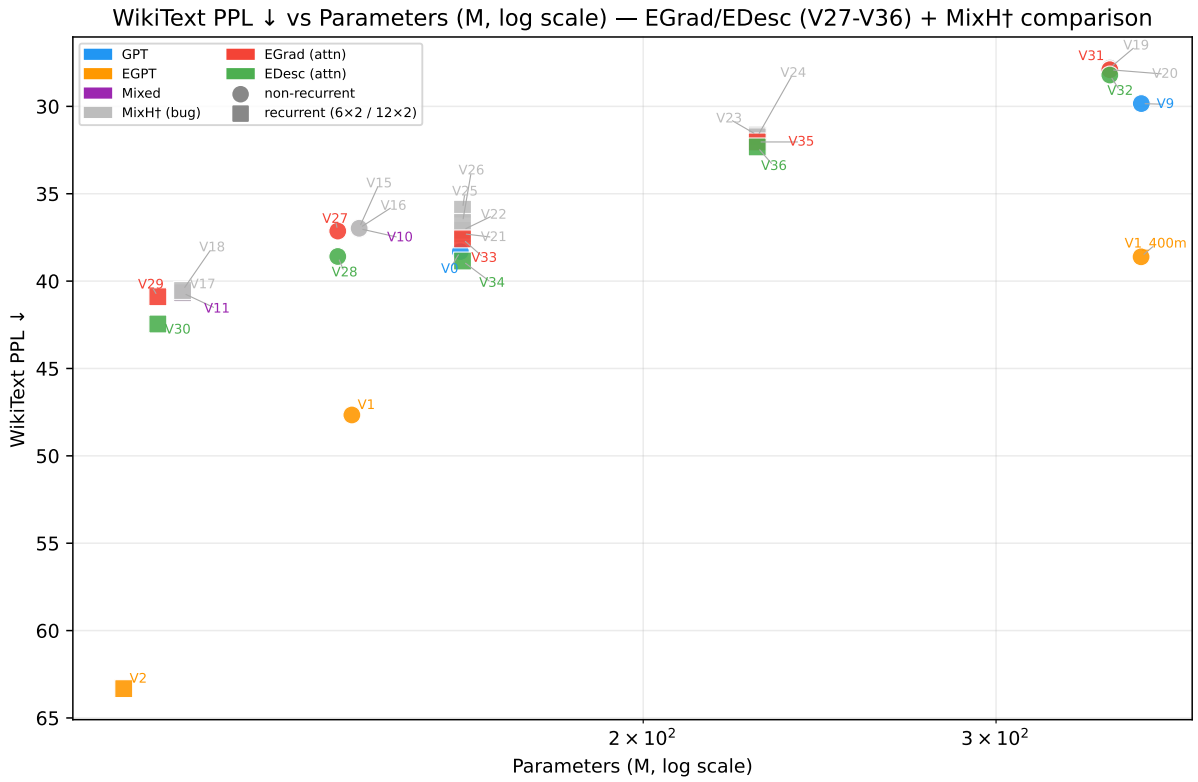


Figure 16: PPL vs. parameters — all runs including MixH† (grey). **Grey points (V15–V26) are mislabeled:** intended as EGrad/EDesc but trained as plain MixedHead due to a config serialization bug (see §2). Shown here for reference only; not used in any main-text figures or conclusions.

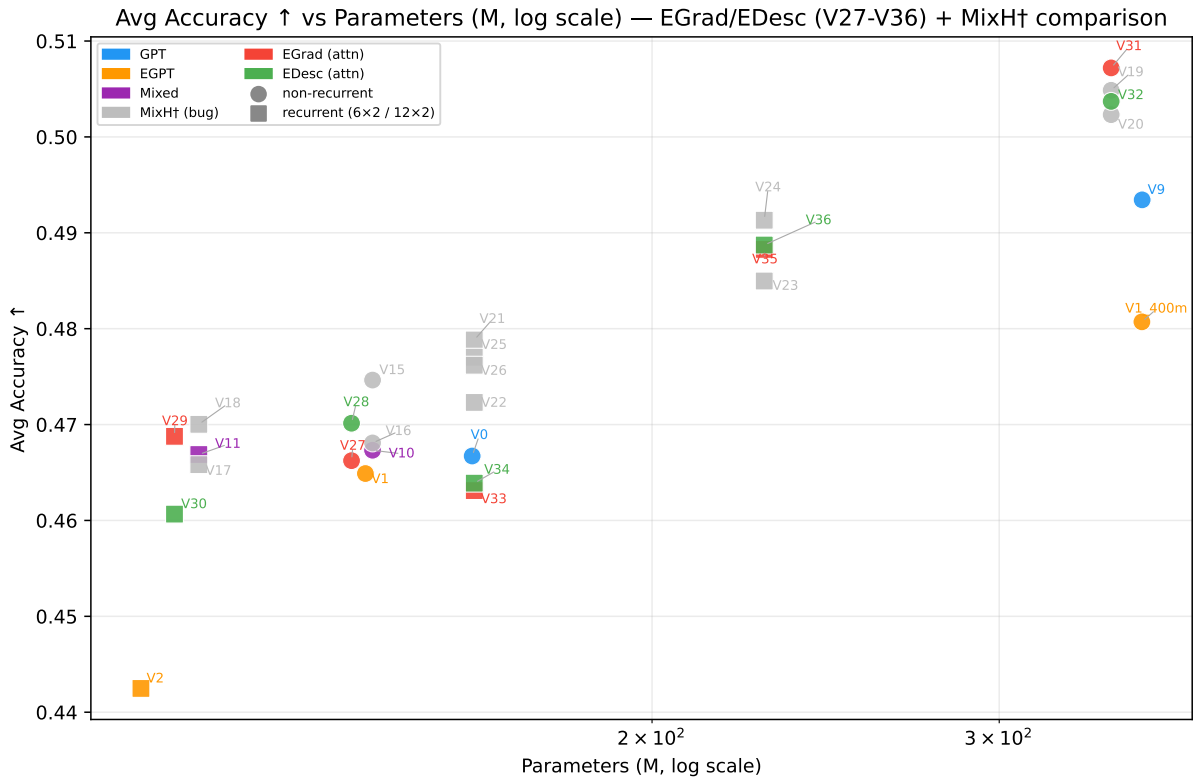
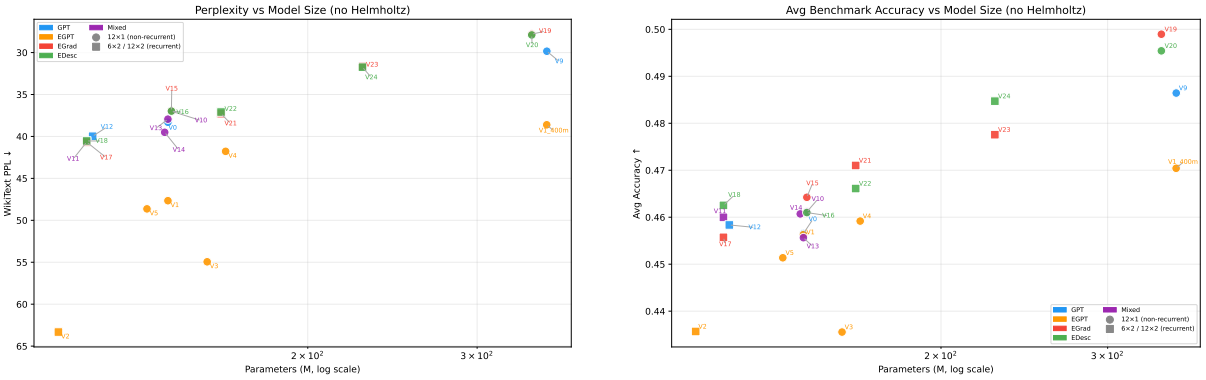


Figure 17: Accuracy vs. parameters — all runs including MixH† (grey). Same caveat as above: grey points are mislabeled runs included for comparison only.



(a) PPL vs. params (log x, no Helmholtz), early runs V0–V18.

(b) Avg accuracy vs. params (log x), early runs V0–V18.

Figure 18: Parameter-efficiency across early variants (V0–V18). Recurrent models (squares) cluster at smaller unique parameter counts.

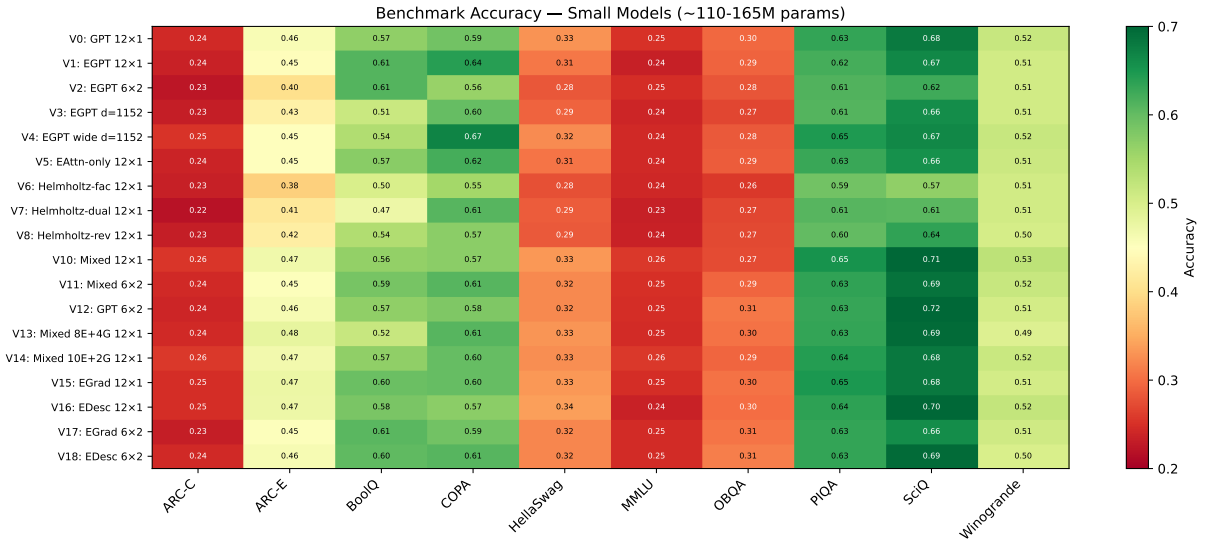


Figure 19: Benchmark accuracy heatmap for all small-scale variants (V0–V18). V15 EGrad and V16 EDesc occupy the top rows alongside V0 GPT.

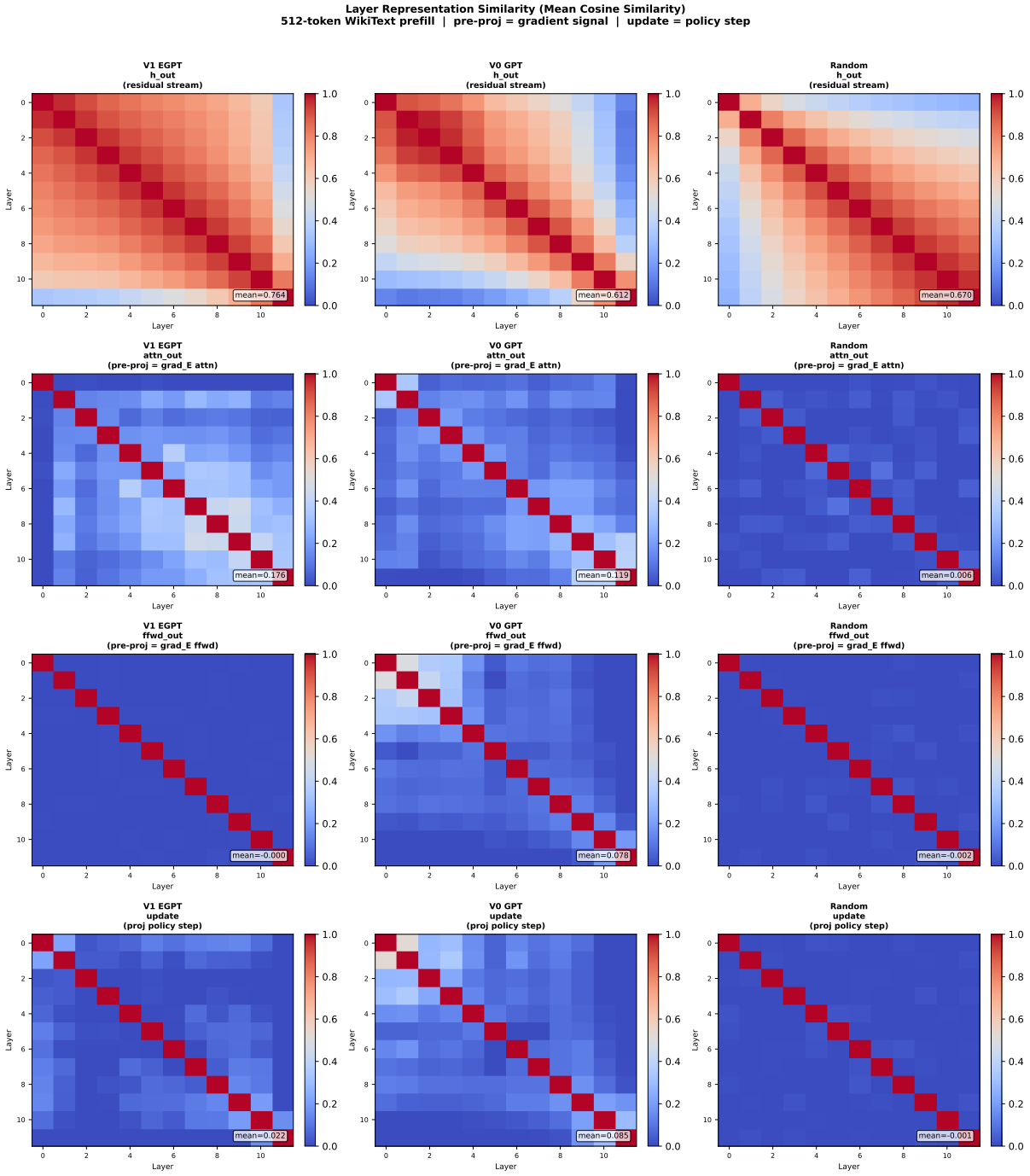


Figure 20: Full 12×12 inter-layer cosine similarity matrices. V1 EGPT attn_out panel shows the highest off-diagonal values.

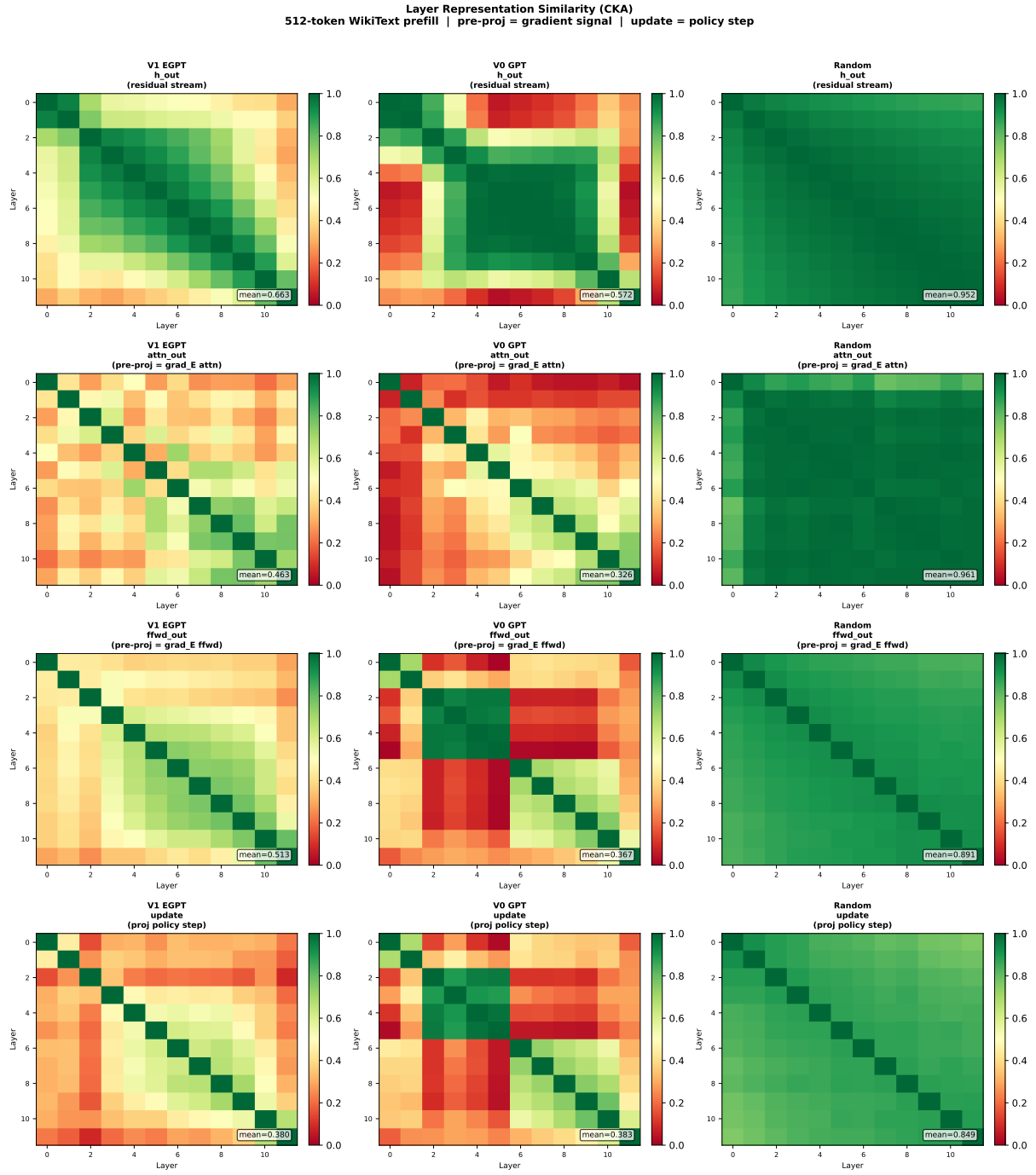
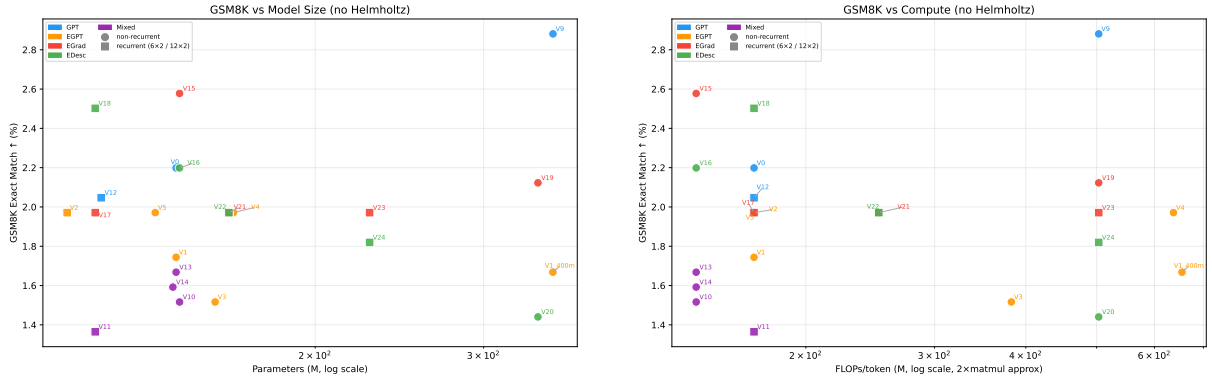


Figure 21: Full 12×12 Linear CKA matrices. The random model has the highest CKA, confirming CKA is dominated by the shared residual stream and is a poor probe.



(a) GSM8K vs. parameters.

(b) GSM8K vs. FLOPs/token.

Figure 22: GSM8K exact-match accuracy across all models. All scores <3%; no architecture consistently leads. V9 GPT (2.9%) and V15 EGrad (2.6%) lead at 354M and 144M scale respectively.